

PAL API Guide

This guide is the compact human reference for PAL's public API. It consolidates behavior shared across PAL objects instead of repeating the same geometry, indexing, wrapping, and compilation rules for each class. For current public signatures, use the generated API Reference. For learning PAL from the beginning, use the Manual. For runnable syntax at a glance, use the Cheatsheet.

Core use

```
import PythonAutomataLibrary as pal
```

PAL starts in **safe mode**. Safe mode checks public operations and provides useful errors. To select fast mode:

```
pal.FastMode()
```

Call `FastMode()` before constructing any PAL model or window object. The first such construction locks the process-wide mode. Fast mode removes safety checks from performance-sensitive paths; it is intended for models already validated in safe mode.

Use `@pal.njit` for model computation:

```
@pal.njit
def Step(grid: pal.AgentGrid):
    ...
```

`pal.njit` wraps Numba `jit` and additionally transforms PAL model code. Among other PAL-specific compilation behavior, annotated PAL arguments allow safe-mode errors in compiled model code to report the model source line.

Construction

Function	Purpose
<code>NewGrid(dimensions, dtype)</code>	Typed lattice values.
<code>NewAgentGrid(dimensions, numAgentProps=0, isStackable=False)</code>	Discrete or continuous agents with optional per-agent properties.
<code>NewPopGrid(dimensions, capacity=None)</code>	Integer population counts on a lattice.
<code>NewPDEgrid(dimensions)</code>	Continuous scalar field with diffusion/advection operations.
<code>NewIList()</code>	Reusable integer query/list container.
<code>NewMultinomial(other=None)</code>	Binomial/multinomial sampler.

Spatial grids use one to three nonzero integer dimensions. Each signed dimension must fit the `int32` range (excluding its minimum value), and the product of absolute axis lengths must not exceed the `int32` maximum. Fractional, nonfinite, zero, oversized, or malformed dimension inputs raise `ValueError` during construction. A **negative dimension enables wrapping on that axis** while its absolute value gives the size. `AgentGrid` alone also accepts `()` for a zero-dimensional nonspatial population:

```
grid = pal.NewAgentGrid((100, 100))    # 100 x 100, no wrapping
grid = pal.NewAgentGrid((-100, 100))   # x wraps, y does not
grid = pal.NewPDEgrid((-100, -100))    # x and y wrap
```

`numAgentProps` is the number of per-agent floating-point properties. It must be a nonnegative integer fitting `AgentGrid`'s `int32` metadata budget (which also reserves entries for spatial dimensions); fractional, nonfinite, negative, or oversized values raise `ValueError`. `isStackable` accepts booleans or `0/1`; other values raise `ValueError`. When false, at most one agent occupies each lattice site. `PopGrid.capacity` is an optional nonnegative total-population limit.

`NewMultinomial(other)` accepts another PAL `Multinomial` in the **same safety mode** and copies its solver configuration, not its current sampling state. An unrelated object raises `TypeError`; mixing safe and fast samplers

raises `ValueError`. Create a new sampler without other to use the default solver.

Shared spatial API

`Grid`, `AgentGrid`, `PopGrid`, and `PDEgrid` deliberately share spatial conventions. Geometry metadata (`xDim`, `yDim`, `zDim`, `nDims`, `wrapX`, `wrapY`, `wrapZ`) is read as attributes; geometry operations such as `ToI(...)` and `Box(...)` are methods.

Geometry

Member	Meaning
<code>len(grid)</code>	Number of lattice sites.
<code>xDim, yDim, zDim</code>	Axis sizes. Missing axes are invalid in safe mode.
<code>nDims</code>	Number of spatial dimensions.
<code>wrapX, wrapY, wrapZ</code>	Whether each axis wraps.
<code>ToI(x, y=-1, z=-1)</code>	Convert coordinates to a linear lattice index.
<code>ItoX(i), ItoY(i), ItoZ(i)</code>	Recover coordinates from a linear index.
<code>InWrapX/Y/Z(value)</code>	Wrap an integer lattice coordinate on that axis, or return <code>-1</code> when outside a nonwrapped axis.

The same geometry names are used in 1D, 2D, and 3D. Supply only coordinates/axes that exist for the grid. `Grid`, `PopGrid`, and `PDEgrid` use `InWrapX/Y/Z` for lattice coordinates; `AgentGrid` distinguishes lattice `InWrapSQX/Y/Z` from continuous `InWrapX/Y/Z`.

Regions and neighborhoods

```
grid.Box(x1, x2)
grid.Box(x1, x2, y1, y2)
grid.Box(x1, x2, y1, y2, z1, z2)

grid.Hood(hood, x)
grid.Hood(hood, x, y)
grid.Hood(hood, x, y, z)
```

`Box` iterates coordinates in the half-open rectangular region `[x1, x2)`, `[y1, y2)`, `[z1, z2)` for the dimensions supplied. It yields an integer in 1D, `(x, y)` in 2D, and `(x, y, z)` in 3D. Bounds must be `int32` integers, and the number of bound pairs must match grid dimensionality. On wrapped axes, out-of-range coordinates are wrapped; on nonwrapped axes, they are skipped. A box wider than a wrapped axis can therefore yield repeated sites. `Hood` maps relative offsets around a lattice position to the same coordinate forms, preserving offset order and duplicates; wrapped coordinates are mapped into the grid and nonwrapped out-of-range coordinates are omitted. Offset tuple lengths must match grid dimensionality. Inside `@pal.njit, unroll=True` requests compile-time neighborhood unrolling without changing the returned coordinates. `Box` and `Hood` are direct-iteration constructs in compiled code: top-level `break` or `continue` from their loop body is unsupported; use explicit range loops when that control flow is needed.

Built-in neighborhoods:

```
pal.MooreHood(dim, excludeCenter=False)
pal.VonNeumannHood(dim, excludeCenter=False)
pal.CircleHood(dim, rad, excludeCenter=False)
```

`MooreHood` contains offsets with each coordinate in `[-1, 1]`. `VonNeumannHood` contains the center and axis-adjacent offsets. `CircleHood` contains integer offsets whose Euclidean distance from the center is at most `rad`; `rad` may be noninteger, but must be finite and nonnegative. Set `excludeCenter=True` to omit the zero offset. All three constructors support dimensions 1-3 and reject other dimension counts with `ValueError`. Each returns a tuple of integer offset tuples; these describe relative positions, not site indices, until mapped through `grid.Hood(...)`.

Grid

```
grid = pal.NewGrid((xDim, yDim), dtype)
```

Supported dtypes are bool, signed/unsigned 8/16/32/64-bit integers, and 32/64-bit floats.

Grid supports linear indexing, coordinate indexing, and NumPy-like slices:

```
grid[i]
grid[x, y]
grid[x, y, z]
grid[:] = 0
region = grid[2:5, 3:8]
```

Slice reads return detached NumPy copies. Scalar and slice reads/writes work in Python and compiled PAL model code. In safe mode, invalid scalar indices and coordinates are rejected; fast mode assumes valid access. Safe mode validates bounds, dimensionality, and whether assigned values are representable by the Grid dtype: boolean grids accept only 0/1; integer grids reject fractional, nonfinite, and out-of-range assignments; floating grids reject nonfinite or out-of-range assignments. Use a wider dtype when a calculation may exceed its current range. A detached slice copy is not a live view of the Grid.

AgentGrid

```
agents = pal.NewAgentGrid((xDim, yDim), numAgentProps=2)
```

PAL represents an agent by an integer handle. Agent properties are stored on the grid and accessed with `agents[agent, property]`.

Population and lifecycle

Operation	Meaning
<code>GetPop()</code>	Number of living agents.
<code>Alive(agent)</code>	Whether an agent handle is alive.
<code>NewAgentSQ(...)</code>	Create an agent at a lattice site.
<code>NewAgent(...)</code>	Create an agent at a continuous position.
<code>Dispose(agent)</code>	Remove an agent.
<code>All(shuffle=False)</code>	Snapshot of all living agents; optionally shuffled.

`All()` has **snapshot semantics**. The returned collection can be iterated while agents are created or disposed without changing the current iteration. `All(shuffle=True)` returns the same population in randomized order.

```
for agent in agents.All():
    if ShouldDie(agent):
        agents.Dispose(agent)
```

Position

Discrete/lattice position:

```
agents.I(agent)
agents.XSQ(agent)
agents.YSQ(agent)
agents.ZSQ(agent)
agents.MoveSQ(agent, x, y, z)
```

Continuous position:

```
agents.X(agent)
agents.Y(agent)
agents.Z(agent)
agents.Move(agent, x, y, z)
```

NewAgentSQ and MoveSQ accept either a linear lattice index or lattice coordinates in every spatial dimension. SQ placement uses the center of the selected cell: after NewAgentSQ(x, y), for example, XSQ(agent)==x and YSQ(agent)==y, while X(agent)==x+0.5 and Y(agent)==y+0.5. Continuous NewAgent and Move instead keep the supplied spatial coordinates, with XSQ/YSQ/ZSQ identifying the containing cell.

Occupancy and queries

```
agents.LastAgent(x, y, z)
agents.AgentsAt(x, y, z)
agents.counts[x, y, z]
```

LastAgent returns the most recently stacked agent at a site, or -1 if the site is empty. Iterate with `for agent in agents.AgentsAt(x, y):` in Python or inside `@pal.njit`. Do not structurally modify the AgentGrid during iteration. Safe mode detects structural changes and raises `RuntimeError`; fast mode omits that check. In Python, `AgentsAt(i)` also accepts a linear site index, including for multidimensional grids; `AgentsAt(x, y)` and `AgentsAt(x, y, z)` use coordinates. This overload is distinct from the continuous-position `AgentsInRadius` query. `counts` exposes lattice occupancy counts and supports linear-site, coordinate, and slice indexing (`agents.counts[i]`, `agents.counts[x, y]`, or `agents.counts[:, :]`). It is a **read-only query view**, not a second mutable population grid. Slice queries return detached arrays of occupancy counts. A nonspatial (`dimensions=()`) AgentGrid has no lattice counts; querying `len(agents.counts)` raises `ValueError`.

By default an AgentGrid is not stackable: at most one agent may occupy a lattice site. Set `isStackable=True` when multiple agents per site are required. Safe mode rejects creation or movement into an occupied site on a nonstackable grid without changing model state.

Radius iteration

`AgentsInRadius(rad, x[, y[, z]], exclude=None)` iterates agents near a continuous point in Python or inside `@pal.njit`. The yielded tuple depends on dimensionality:

```
for agent, dx in agents1.AgentsInRadius(rad, x):
    ...
for agent, dx, dy, distSq in agents2.AgentsInRadius(rad, x, y):
    ...
for agent, dx, dy, dz, distSq in agents3.AgentsInRadius(rad, x, y, z):
    ...
```

The radius is inclusive: agents with distance exactly `rad` are included. Returned displacements point from the query center to the agent and use the shortest periodic displacement on wrapped axes; `distSq` is their squared Euclidean distance in 2D/3D. `exclude` omits a specified agent handle. As with Python-side `AgentsAt`, structural modification during iteration is unsupported, and the generation check is enabled only in safe mode.

Wrapping

Discrete coordinates:

```
InWrapSQX(value)
InWrapSQY(value)
InWrapSQZ(value)
```

Continuous coordinates:

```
InWrapX(value)
InWrapY(value)
InWrapZ(value)
```

Wrapped displacement:

```
DispWrapX(x1, x2)
DispWrapY(y1, y2)
DispWrapZ(z1, z2)
```

`InWrapSQ*` maps integer lattice coordinates into the corresponding axis; `InWrap*` does the same for continuous coordinates. On a nonwrapped axis, an out-of-range coordinate returns -1. `DispWrap*(x1, x2)` returns the shortest signed displacement from `x1` to `x2`, accounting for periodic boundaries.

PopGrid

```
pop = pal.NewPopGrid((xDim, yDim), capacity=None)
```

PopGrid stores integer population counts. Indexing and direct assignment modify the current population immediately.

```
pop[x, y]
pop[x, y] = 10
pop[:] = 0
```

Linear (`pop[:]`) and spatial (`pop[:, :]`) slice reads return detached `int64` NumPy arrays; slice assignment writes to the population. Population changes can instead be accumulated and applied together:

```
pop.Add(delta, x, y)
pop.Update()
```

Add changes the pending delta, not the current value: `GetPop()`, indexing, and `All()` continue to report the current population until `Update()`. `Update()` applies all pending changes simultaneously. This is useful when a timestep should not depend on iteration order.

Operation	Meaning
<code>GetPop()</code>	Total population over all sites.
<code>All()</code>	Copy of the linear indices of currently nonzero sites.
<code>Reset()</code>	Clear current populations and pending changes.

`capacity=None` uses the `int64` maximum as the per-site population cap. An explicit `capacity` must be a nonnegative `int64` integer; invalid, fractional, nonfinite, or out-of-range values raise `ValueError` during construction. Capacity constrains each site independently; total population is tracked separately and must remain representable as a nonnegative `int64`. In safe mode, direct writes enforce these bounds immediately. `Update()` validates every resulting site count and the total population before applying any pending delta; if validation fails, current populations and pending deltas are unchanged. Fast mode assumes these constraints are satisfied.

PDEgrid

```
field = pal.NewPDEgrid((xDim, yDim))
```

PDEgrid stores a continuous `float32` scalar field. Direct indexing/assignment changes the current field immediately. Slice reads return detached `float32` NumPy arrays; array slice assignment writes to the field, but editing a previously read slice does not. Linear (`field[:]`) and spatial (`field[:, :]`) slices are supported. `Add(value, ...)` accumulates a pending delta; `Update()` applies pending changes simultaneously. `Reset()` clears both the field and pending changes.

Space and time

```
field.SetTimeSpaceStep(dt, dx, dy=1.0, dz=1.0)
field.Dt()
field.Dx()
field.Dy()
field.Dz()
```

Transport methods use these spacings. `Dy()` and `Dz()` apply only when those dimensions exist; safe mode rejects missing-dimension access. `dt`, `dx`, `dy`, and `dz` must be finite and positive in safe mode.

Diffusion

```
field.Diffusion(rateConstant, ...)
field.DiffusionMask(rateConstant, mask)
field.DiffusionField(rateConstants, ...)
field.DiffusionInterfaces(rateConstantsX, rateConstantsY=None, rateConstantsZ=None, ...)
field.DiffusionADI(rateConstant, ...)
field.DiffusionRadialCircle(rateConstant, outerBC=None)
field.DiffusionRadialSphere(rateConstant, outerBC=None)
```

Diffusion uses one constant diffusion rate. `DiffusionField` uses spatially varying cell-centered rates; each interior face uses the harmonic mean of its two adjacent cell rates, so a zero rate on either side blocks diffusion across that face. `DiffusionInterfaces` supplies the positive-face interface rate for each cell and axis directly; the negative face uses the neighboring cell's stored positive-face rate. `DiffusionMask` treats mask values ≥ 0 as active and values < 0 as blocked; diffusion occurs only between adjacent active cells, so the magnitude of a nonnegative mask value is irrelevant. `DiffusionADI` uses Crank-Nicolson in 1D, Peaceman-Rachford ADI in 2D, and Douglas-Gunn directional solves in 3D. `DiffusionRadialCircle` and `DiffusionRadialSphere` use radial shell finite-volume discretizations on a nonwrapped 1D grid with at least two cells. Cell i spans radii $i \cdot dx$ to $(i+1) \cdot dx$, so cell 0 is the central disk/sphere and symmetry gives zero flux through its inner face. `outerBC` optionally sets the exterior concentration at the outer face.

Cartesian diffusion methods accept optional `xMinBC`, `xMaxBC`, `yMinBC`, `yMaxBC`, `zMinBC`, and `zMaxBC` fixed boundary values. A boundary may be a scalar applied uniformly across the face or a C-contiguous `float32` array with one value per site on that face; for example, an x-face array has `yDim*zDim` values in 3D. On a nonwrapped face, `None` means no flux through that face; wrapped axes are periodic and do not accept explicit boundary conditions. Field/interface arrays must match the grid geometry and dimensionality. Safe mode validates boundary size/type/finite values, explicit-scheme stability, and other inputs before changing pending state.

Advection

```
field.Advection(vx, vy=0.0, vz=0.0, ...)
field.AdvectionField(xVels, yVels=None, zVels=None, ...)
field.AdvectionInterfaces(xVels, yVels=None, zVels=None, ...)
```

Advection uses a conservative first-order upwind discretization. `Advection` uses constant velocity components. `AdvectionField` uses cell-centered velocity fields and averages adjacent values at interior faces. `AdvectionInterfaces` supplies positive-face velocities directly; the negative face uses the neighboring cell's stored positive-face velocity. Supply velocity components only for dimensions that exist. The same scalar-or-face-array Cartesian boundary arguments used by diffusion are available: `None` closes a nonwrapped face to advective flux, while supplied values provide the exterior concentration used for inflow. Safe mode checks the CFL condition before accumulating changes.

IList

```
items = pal.NewIList()
```

`IList` is PAL's reusable integer query/list container. Stored values are nonnegative `int32` integers in the supported contract.

Operation	Meaning
<code>Append(i)</code>	Append a nonnegative <code>int32</code> value; returns the <code>IList</code> for chaining.
<code>Clear()</code>	Remove all entries; returns the <code>IList</code> for chaining.
<code>Random()</code>	Return a random entry.
<code>Shuffle()</code>	Shuffle entries in place; returns the <code>IList</code> for chaining.
<code>All()</code>	Return a detached copy.
<code>Iter()</code>	Return a no-copy/live iterable view.
<code>len(items), items[i]</code>	Length/index access.

Use `Iter()` for one-pass iteration when a copy is unnecessary. Use `All()` when the returned values must remain independent of subsequent `IList` changes.

Random numbers and Multinomial

```
pal.Seed(seed)
pal.Random()      # uniform float
pal.RandInt(max)  # integer in [0, max)
```

Seed accepts integral seeds in the uint64 domain (0 through $2^{64} - 1$); invalid or nonfinite inputs raise `ValueError`. `RandInt(max)` requires a positive integer in the int64 domain and returns an integer in $[0, \text{max})$; zero, negative, fractional, nonfinite, or out-of-range bounds raise `ValueError`. `Random()` returns a uniform floating-point draw. PAL random functions share one seeded stream across Python and compiled PAL calls, so reseeding reproduces the same call sequence across that boundary. The stream is call-order dependent: adding or removing a random draw changes later results. This does not seed NumPy's separate RNG.

For binomial/multinomial sampling:

```
multi = pal.NewMultinomial()
x = multi.Binomial(n, p)

multi.Setup(n)
a = multi.Sample(pA)
b = multi.Sample(pB)
# remaining count/probability mass stays in the sampler
```

`Setup(n)` initializes a multinomial sequence with n trials and probability mass 1 and is chainable. Each `Sample(p)` takes that category's probability in the original multinomial distribution, draws from the remaining trials using the corresponding conditional probability, then removes the sampled count and p from the remaining state. Thus successive calls such as `Sample(0.2)` and `Sample(0.3)` represent categories with probabilities 0.2 and 0.3, not 0.2 followed by 0.3 of the remainder; any unrequested probability mass remains unsampled. In safe mode cumulative requested probability cannot exceed 1. `Binomial(n, p)` performs an independent binomial draw and does not consume the current multinomial sequence. `NewMultinomial(other)` copies solver configuration but starts with fresh sampling state.

Visualization

PAL provides a pixel renderer for lattice models and an OpenGL renderer for 2D/3D geometry. Rendering runs separately from model computation.

PixWindow

```
pix, window = pal.StartPixWindow(xDim, yDim, scale=1, title="PAL", headless=False)
```

`pix` is a write-oriented pixel buffer, not a general-purpose numeric `Grid`: it supports assignment but does not expose a public pixel-read API. Assign integer RGB colors by coordinate or linear index:

```
pix[x, y] = 0xFF0000
pix[i] = 0x00FF00
```

It also provides `Xdim()`, `Ydim()`, `ToI(x, y)`, `ItoX(i)`, `ItoY(i)`, and `len(pix)`.

The window lifecycle is:

```
window.Update()
window.Save(path, block=False)
window.StartGif(path, delay=100)
window.AddGifFrame(block=False)
window.StopGif()
window.IsOpen()
window.Close()
```

`Update()` publishes the current pixel buffer to the renderer. `StartGif()` begins a GIF recording, each `AddGifFrame()` appends the currently rendered frame, and `StopGif()` finalizes the file. `block=True` on asynchronous output operations waits for that requested output to complete before returning; call `StopGif()` before relying on the GIF file as complete.

OpenGLWindow

```
draw, window = pal.StartOpenGLWindow(xDim, yDim, zDim=None,
                                     width=800, height=800,
                                     title="PAL", headless=False)
```

Draw geometry with:

```
draw.Circle(rad, color, x, y, z=0.0)
draw.Box(xLen, color, x, y, z=0.0, yLen=None, zLen=None)
```

```
draw.BoxSQ(color, x, y, z=0.0)
draw.Line(width, color, x1, y1, x2, y2, z1=0.0, z2=0.0)
```

Window controls include `Borders(width, color)`, `Camera(x, y, z, yaw=None, pitch=None)`, `Background(color)`, `Clear()`, `Update()`, `IsOpen()`, `Save(path, block=False)`, `StartGif(path, delay=100)`, `AddGifFrame(block=False, timeout=30)`, `StopGif(timeout=30)`, and `Close()`. The GIF lifecycle is the same as for `PixWindow`: start recording, append rendered frames, then stop to finalize the file.

Colors are packed integer RGB values such as `0xFF0000`. `pal.ColorScale(colors, value)` interpolates RGB channels across a sequence of colors for normalized value; values outside `[0, 1]` clamp to the endpoint colors.

Set `headless=True` for off-screen image/GIF generation. OpenGL headless rendering requires a supported standalone backend such as EGL.

Use `pal.AwaitWindows()` when model execution should wait for PAL windows to finish their lifecycle.

Safe and fast contracts

Safe and fast objects expose the same modeling API. PAL starts in safe mode. To opt into fast mode, call `pal.FastMode()` **before constructing any PAL model or window object**. The first such construction locks the process-wide mode selection; a later `pal.FastMode()` raises `RuntimeError`. Run safe and fast comparisons in separate processes rather than trying to switch modes mid-run. Safe mode validates public operations and is the development/default mode; fast mode assumes the model obeys those contracts and removes performance-sensitive checks. Examples of safe-mode checks include lattice bounds and dimensionality, dead-agent and occupancy errors, `PopGrid` overflow, and PDE field/interface array shape or dimensionality. Invalid operations in fast mode are outside the supported contract rather than an alternative error-handling API.

Use safe mode until the model is correct, then benchmark fast mode. Do not rely on safe-mode exceptions as model control flow.

Saving PAL objects

PAL native-backed model objects support standard Python pickling. Checkpoints preserve the state needed to resume the object rather than process-local pointers: `AgentGrid` geometry, occupancy, agents and properties; `PopGrid` geometry, capacity, current counts and pending deltas; `PDEgrid` geometry, time/space steps, current field and pending deltas; `IList` contents; and a `Multinomial`'s current `Setup/Sample` remainder. Safe/fast identity is also restored. This means a `PopGrid` or `PDEgrid` may be pickled before `Update()` without losing its queued changes.

Choosing the right structure

Need	PAL structure
One typed value per lattice site	<code>Grid</code>
Individual agents with identity/properties	<code>AgentGrid</code>
Integer counts without individual identity	<code>PopGrid</code>
Continuous diffusing/advection field	<code>PDEgrid</code>
Reusable integer query result	<code>IList</code>
Binomial/multinomial draws	<code>Multinomial</code>

These structures are designed to be composed inside the same compiled model step. A model can, for example, query agents, accumulate population counts, update a PDE field, and feed the resulting field back into agent behavior without leaving `@pal.njit`.